

Título del trabajo: “Algoritmos de Búsqueda y Ordenamiento en Python”

• **Alumnos:**

Cabrera Ezequiel Dario - dario.cabrera17@gmail.com

Bonetti Kunt Daniela Sofia - danielabonetti51@gmail.com

• **Materia:** Programación I

• **Profesor/a:** Prof. Cinthia Rigoni

• **Fecha de Entrega:** 09/06/2025

Índice

1. Introducción
2. Marco Teórico
3. Caso Práctico
4. Metodología Utilizada
5. Resultados Obtenidos
6. Conclusiones
7. Bibliografía
8. Anexos

1. Introducción

Los algoritmos de búsqueda y ordenamiento son fundamentales en el desarrollo de software, ya que permiten la gestión eficiente de grandes volúmenes de datos y optimizan el rendimiento de las aplicaciones. Su correcta implementación resulta clave en diversos ámbitos, como la administración de bases de datos y el desarrollo de sistemas inteligentes. Sin estos algoritmos, muchas tareas cotidianas en software serían lentas y poco optimizadas, afectando el rendimiento y la experiencia del usuario.

Este trabajo tiene como objetivo analizar y aplicar distintos métodos de búsqueda y ordenamiento en Python, comparando su desempeño en distintos escenarios y evaluando su impacto en la organización y manipulación de datos. A través de una investigación práctica, se busca comprender en profundidad los principios que rigen estos algoritmos y desarrollar implementaciones que permitan analizar su eficiencia mediante pruebas comparativas. La programación moderna se basa en la optimización, y elegir el algoritmo adecuado puede marcar la diferencia entre una solución eficiente y una que consume demasiados recursos.

Además del estudio teórico, este trabajo incluirá la implementación práctica de diferentes algoritmos en Python. Se desarrollarán pruebas comparativas para medir su eficiencia y analizar el impacto que tienen en el manejo de datos. Estos resultados permitirán comprender qué métodos son más adecuados para distintos escenarios y cómo pueden aplicarse en proyectos reales.

2. Marco Teórico

Definiciones

Búsqueda

La búsqueda es una operación fundamental en programación que se utiliza para encontrar un elemento específico dentro de un conjunto de datos. Es una tarea común en muchas aplicaciones, como bases de datos, sistemas de archivos y algoritmos de inteligencia artificial.

Comprender y utilizar correctamente los distintos métodos de búsqueda mejora el rendimiento y eficiencia de los programas.

Ordenamiento

El ordenamiento organiza los datos de acuerdo a un criterio, como de menor a mayor o alfabéticamente. Los algoritmos de ordenamiento son importantes porque permiten organizar y estructurar datos de manera eficiente. Al ordenar los datos, se pueden realizar búsquedas, análisis y otras operaciones de manera más rápida y sencilla.

Clasificación

Algunos de los algoritmos de búsqueda más comunes son:

- **Búsqueda lineal:** Es el algoritmo de búsqueda más simple, que recorre cada elemento del conjunto de datos de forma secuencial hasta encontrar el elemento deseado. Es fácil de implementar, pero puede ser lento para conjuntos de datos grandes.
- **Búsqueda binaria:** Es un algoritmo de búsqueda eficiente que funciona en conjuntos de datos ordenados. Divide el conjunto de datos en dos mitades y busca el elemento deseado en la mitad correspondiente. Repite este proceso hasta encontrar el elemento o determinar que no está en el conjunto de datos.
- **Búsqueda de interpolación:** Es un algoritmo de búsqueda que mejora la búsqueda binaria al estimar la posición del elemento deseado en función de su valor. Puede

ser más eficiente que la búsqueda binaria para conjuntos de datos distribución uniforme de valores.

- Búsqueda de hash: Es un algoritmo de búsqueda que utiliza una función hash para asignar cada elemento a una ubicación única en una tabla hash. Esto permite acceder a los elementos en tiempo constante, lo que lo hace muy eficiente para conjuntos de datos grandes.

Algunos de los algoritmos de ordenamiento más comunes incluyen:

- Ordenamiento por burbuja: Es un algoritmo de ordenamiento simple y fácil de implementar. Funciona comparando cada elemento de la lista con el siguiente elemento y luego intercambiando los elementos si están en el orden incorrecto.
- Ordenamiento por selección: Es otro algoritmo de ordenamiento simple que funciona encontrando el elemento más pequeño de la lista y luego intercambiándolo con el primer elemento. Este proceso se repite hasta que todos los elementos de la lista estén ordenados.
- Ordenamiento por inserción: Es un algoritmo de ordenamiento que funciona insertando cada elemento de la lista en su posición correcta en la lista ordenada.
- Ordenamiento rápido: Es un algoritmo de ordenamiento eficiente que funciona dividiendo la lista en dos partes y luego ordenando cada parte de forma recursiva.
- Ordenamiento por mezcla: Es un algoritmo de ordenamiento eficiente que funciona dividiendo la lista en dos partes, ordenando cada parte y luego fusionando las dos partes ordenadas.

Jerarquías

1. Jerarquía de Estructuras de Datos

Algunas formas de organizar la información son jerárquicas, como un árbol familiar. Esto significa que los datos se organizan en niveles para que sean más fáciles y rápidos de encontrar o manejar. Por ejemplo, un árbol binario organiza los datos de forma que las búsquedas sean muy rápidas, ya que en cada paso se elimina la mitad de las opciones.

También, estructuras como los Heaps se usan en algoritmos como el elemento más grande o más pequeño siempre accesible, lo que



2. Jerarquía de Eficiencia de Algoritmos (Notación Big O)

La segunda forma de entender la jerarquía es comparando qué tan rápidos y eficientes son los algoritmos cuando la cantidad de datos crece. Usamos la Notación Big O para clasificar su rendimiento. Aquí está la jerarquía de eficiencia principal, de los algoritmos más rápidos a los más lentos para grandes volúmenes de datos:

- $O(1)$ - Constante: Siempre tarda el mismo tiempo, sin importar cuántos datos haya.
- $O(\log n)$ - Logarítmico: Muy eficiente. El tiempo crece muy, muy despacio al aumentar los datos. La Búsqueda Binaria es un ejemplo.
- $O(n)$ - Lineal: El tiempo crece de forma directa con la cantidad de datos. La Búsqueda Lineal funciona así.
- $O(n \log n)$ - Lineal-Logarítmico: Muy buena para ordenar muchos datos. QuickSort y MergeSort suelen tener esta eficiencia.
- $O(n^2)$ - Cuadrático: Se vuelven muy lentos rápidamente con muchos datos. Bubble Sort o Selection Sort son ejemplos de esto, simples pero ineficientes para grandes volúmenes.

Entender esta jerarquía de eficiencia es crucial para elegir el algoritmo adecuado, asegurando que nuestras aplicaciones funcionen de manera óptima con cualquier cantidad de información.

Estructura

1. Definición del problema

Se establece qué se necesita resolver (buscar un elemento, ordenar datos).

2. Entrada de datos

Se define la estructura a utilizar (listas, diccionarios, árboles).

Se carga la información sobre la cual operará el algoritmo.

3. Inicialización de variables

Se establecen valores iniciales, como índices o límites de búsqueda.

4. Iteración o recursión

Se repiten procesos hasta que la condición de salida se cumpla.

Algunos algoritmos usan bucles (for, while), mientras que otros usan recursión (llamadas sucesivas a la misma función).

5. Condiciones de salida

Se determina cuándo debe detenerse la ejecución, por ejemplo, al encontrar un elemento o cuando la lista está completamente ordenada.

6. Retorno del resultado

Se devuelve la posición de un elemento (en búsqueda) o la lista ordenada (en ordenamiento).

Se puede mostrar el resultado en pantalla o guardarlo en un archivo.

Sintaxis

Ejemplo de Sintaxis de Búsqueda Lineal

```
def busqueda_lineal(lista, objetivo):  
  
    for i in range(len(lista)):  
  
        if lista[i] == objetivo:  
  
            return i  
  
    return -1
```

Ejemplo de Sintaxis de Búsqueda Binaria

```
def busqueda_binaria(lista, objetivo):  
  
    izquierda, derecha = 0, len(lista) - 1  
  
    while izquierda <= derecha:  
  
        medio = (izquierda + derecha) // 2  
  
        if lista[medio] == objetivo:  
  
            return medio  
  
        elif lista[medio] < objetivo:  
  
            izquierda = medio + 1  
  
    else:
```

```
derecha = medio - 1  
  
return -1
```

Ejemplo de Sintaxis de ordenamiento Burbuja

```
import random  
  
import timeit  
  
def bubble_sort(arr):  
  
    n = len(arr)  
  
    for i in range(n):  
  
        for j in range(0, n - i - 1):  
  
            if arr[j] > arr[j + 1]:  
  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
# Generar una lista aleatoria de tamaño 1000  
  
data = [random.randint(1, 1000) for _ in range(10000)]  
  
data_little = [5,4,23,2,1,45,212,32,54]  
  
# Medir el tiempo de ejecución para grandes sets de datos  
  
start_time = timeit.default_timer()  
  
bubble_sort(data_little)  
  
end_time = timeit.default_timer()
```

```
print("Tiempo de ejecución para lista pequeña:", end_time - start_time,
      "segundos")

# Medir el tiempo de ejecución para grandes sets de datos

start_time = timeit.default_timer()

bubble_sort(data)

end_time = timeit.default_timer()

print("Tiempo de ejecución para lista grande:", end_time - start_time,
      "segundos")
```

Ejemplo de Sintaxis de Ordenamiento por selección

```
import random

import timeit

def selection_sort(arr):

    n = len(arr)

    for i in range(n):

        # Encontrar el índice del elemento mínimo

        min_index = i

        for j in range(i+1, n):

            if arr[j] < arr[min_index]:

                min_index = j

        # Intercambiar el elemento mínimo con el elemento actual

        arr[i], arr[min_index] = arr[min_index], arr[i]
```

```
# Lista no ordenada
my_list = [5, 4, 23, 2, 1, 45, 212, 32, 54]

# Medir tiempo para lista pequeña
start_time = timeit.default_timer()
selection_sort(my_list.copy())
end_time = timeit.default_timer()
print("Tiempo para una lista pequeña:", end_time - start_time)

# Medir tiempo para lista grande
start_time = timeit.default_timer()
selection_sort(random_list.copy())
end_time = timeit.default_timer()
print("Tiempo para una lista grande:", end_time - start_time)
```

Ejemplo de Sintaxis de Quicksort:

```
def quicksort(arr):
    if len(arr) <= 1:
        return arr
    else:
        pivot = arr[0]
        less = [x for x in arr[1:] if x <= pivot]
        greater = [x for x in arr[1:] if x > pivot]
        return quicksort(less) + [pivot] + quicksort(greater)

# Lista pequeña
```

```
my_list = [5, 4, 23, 2, 1, 45, 212, 32, 54]

# Medir tiempo para lista pequeña

start_time = timeit.default_timer()

quicksort(my_list.copy())

end_time = timeit.default_timer()

print("Tiempo para una lista pequeña:", end_time - start_time)

# Medir tiempo para lista grande

start_time = timeit.default_timer()

quicksort(random_list.copy())

end_time = timeit.default_timer()

print("[QUICKSORT] Tiempo para una lista grande:", end_time -
start_time)

# Medir tiempo para lista grande con seleccion

start_time = timeit.default_timer()

selection_sort(random_list.copy())

end_time = timeit.default_timer()

print("[SELECTION] Tiempo para una lista grande:", end_time -
start_time)

# Medir tiempo para lista grande con insertion

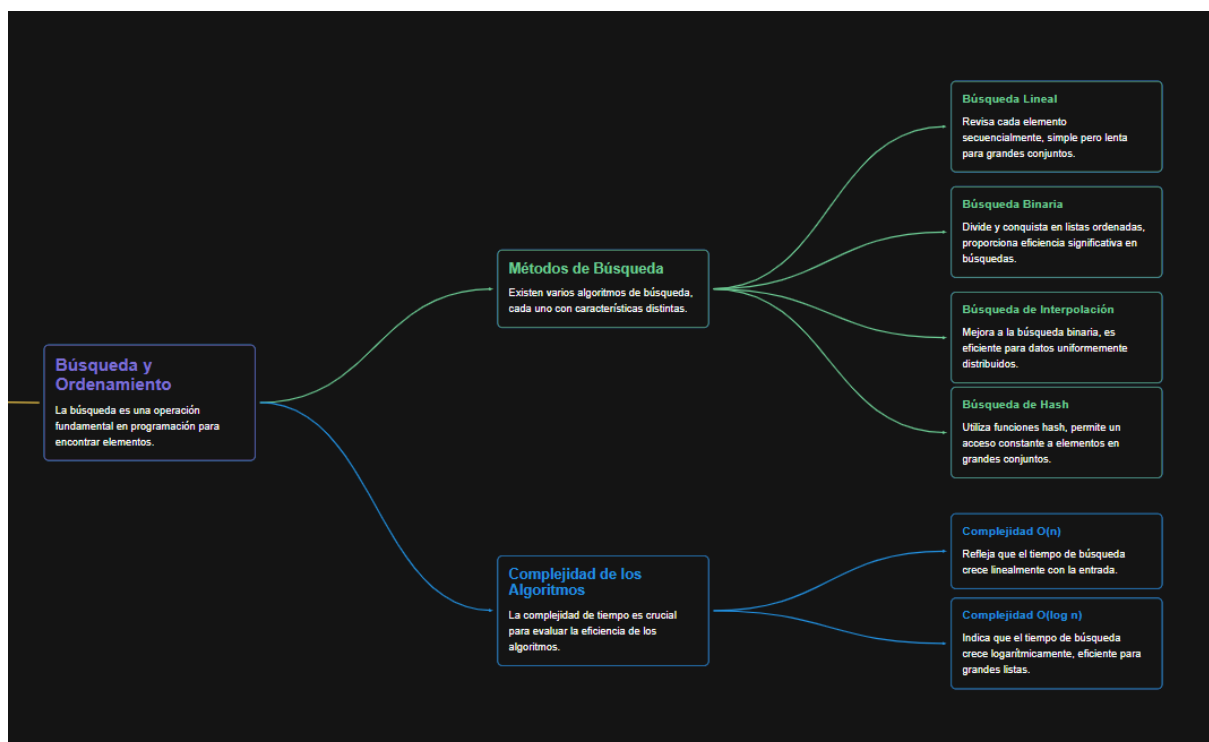
start_time = timeit.default_timer()

insertion_sort(random_list.copy())
```

```
end_time = timeit.default_timer()

print("[INSERTION] Tiempo para lista grande :", end_time - start_time)
```

Gráficos, esquemas o tablas





Algoritmo	Descripción	Ventajas	Desventajas
Burbuja (Bubble Sort)	Compara elementos adyacentes y los intercambia si están en el orden incorrecto, repitiendo hasta que no haya más intercambios.	Fácil de entender e implementar.	Muy ineficiente para listas grandes.
Selección (Selection Sort)	Encuentra el elemento más pequeño (o grande) de la parte no ordenada y lo coloca al inicio de la parte ordenada.	Simple de implementar, estable.	Lento para listas grandes.
Inserción (Insertion Sort)	Construye la lista ordenada elemento por elemento, insertando cada nuevo elemento en su posición correcta en la parte ya ordenada.	Eficiente para listas pequeñas o casi ordenadas.	Ineficiente para listas grandes y desordenadas.
Rápido (Quick Sort)	Selecciona un "pivote" y particiona la lista, colocando los elementos menores a un lado y los mayores al otro, luego ordena recursivamente las sublistas.	Generalmente muy rápido en la práctica.	El peor caso de complejidad es $O(n^2)$ (aunque es raro en la práctica).
Mezcla (Merge Sort)	Divide la lista a la mitad repetidamente hasta que cada sublista tenga un solo elemento, luego fusiona las sublistas ordenadamente.	Eficiente y estable, garantiza un tiempo de ejecución $O(n \log n)$.	Requiere espacio adicional para las sublistas fusionadas.

Cuadro creado por Cabrera Dario Ezequiel/ Bonetti Daniela Sofia

Algoritmo	Descripción	Ventajas	Desventajas	Complejidad (Peor Caso)
Lineal	Recorre cada elemento secuencialmente hasta encontrar el deseado o llegar al final de la lista.	Fácil de implementar, simple.	Muy lento para conjuntos de datos grandes.	$O(n)$
Binaria	Requiere que los datos estén ordenados. Divide el conjunto de datos a la mitad repetidamente hasta encontrar el elemento.	Muy eficiente para conjuntos de datos grandes.	Requiere que los datos estén previamente ordenados, lo que puede consumir tiempo.	$O(\log n)$
Interpolación	Mejora la búsqueda binaria estimando la posición del elemento según su valor, útil en distribuciones uniformes.	Más eficiente que la binaria para datos grandes con distribución uniforme.	Rendimiento variable si los datos no están uniformemente distribuidos.	$O(\log \log n)$ en promedio, $O(n)$ en peor caso
Hash	Utiliza una función hash para asignar cada elemento a una ubicación única en una tabla hash, permitiendo acceso directo.	Acceso en tiempo constante, muy eficiente para grandes conjuntos.	Requiere una buena función hash; las colisiones pueden degradar el rendimiento.	$O(1)$ en promedio, $O(n)$ en peor caso

Cuadro creado por Cabrera Dario Ezequiel/ Bonetti Daniela Sofia

Documentación

Para el desarrollo de este Marco teórico nos apoyamos principalmente en el material compartido en el aula virtual. Utilizamos el PDF sobre el tema desarrollado, también fue material audiovisual compartido por los profesores donde logramos entender mejor aún conceptos técnicos del tema. Por otro lado, también utilizamos el archivo .ipynb que nos compartieron en el aula para el desarrollo de la sintaxis.

- [Búsqueda y Ordenamiento en Programación], "Búsqueda y Ordenamiento en Programación".
- [Nicolas Quiroz], "ORDENAMIENTO", [Plataforma: Aula Virtual/Youtube]. Link: <https://www.youtube.com/watch?v=xntUhrhtLaw>
- [Nicolas Quiroz], "BUSQUEDA", [Plataforma: Aula Virtual/Youtube]. Link: <https://www.youtube.com/watch?v=gJIQTq80Ilg>
- [BusquedaOrdenamiento.ipynb].

3. Caso Práctico

El objetivo de este caso práctico es mostrar, con pruebas, cómo la elección de un algoritmo de búsqueda u ordenamiento afecta el tiempo que tarda un programa en funcionar, especialmente cuando los datos son muchos. Para esto, simularemos cómo se procesan listas de números, comparando el rendimiento de varios algoritmos. Así veremos en la práctica cómo se comportan y si coinciden con lo que dice la teoría.

Archivo: Algoritmos.py

a. Algoritmos de búsqueda:

Búsqueda Lineal

```
def busqueda_lineal (lista, objetivo):  
    for i in range(len(lista)):  
        if lista[i] == objetivo:  
            return i  
    return -1
```

Búsqueda Binaria

```
def busqueda_binaria (lista_ordenada, elemento_objetivo):  
    izquierda = 0  
    derecha = len(lista_ordenada) - 1  
  
    while izquierda <= derecha:  
        medio = (izquierda + derecha) // 2  
  
        if lista_ordenada[medio] == elemento_objetivo:  
            return medio  
        elif lista_ordenada[medio] < elemento_objetivo:  
            izquierda = medio + 1  
        else:  
            derecha = medio - 1  
  
    return -1
```

Búsqueda Interpolación

```
def busqueda_interpolacion (lista_ordenada, elemento_objetivo):  
    izquierda = 0  
    derecha = len(lista_ordenada) - 1  
  
    while izquierda <= derecha and lista_ordenada[izquierda] <= elemento_objetivo <= lista_ordenada[derecha]:  
        if izquierda == derecha:  
            if lista_ordenada[izquierda] == elemento_objetivo:  
                return izquierda  
            return -1  
  
        denominador = lista_ordenada[derecha] - lista_ordenada[izquierda]  
        if denominador == 0:  
            if lista_ordenada[izquierda] == elemento_objetivo:  
                return izquierda  
            else:  
                return -1  
  
        pos = izquierda + ((elemento_objetivo - lista_ordenada[izquierda]) * \  
            (derecha - izquierda) // denominador)  
  
        if lista_ordenada[pos] == elemento_objetivo:  
            return pos  
        elif lista_ordenada[pos] < elemento_objetivo:  
            izquierda = pos + 1  
        else:  
            derecha = pos - 1  
    return -1
```

b. Algoritmos de Ordenamiento:

Ordenamiento Burbuja (Bubble Sort)

```
def bubble_sort(lista):  
    n = len(lista)  
    for i in range(n):  
        for j in range(0, n - i - 1):  
            if lista[j] > lista[j + 1]:  
                lista[j], lista[j + 1] = lista[j + 1], lista[j]  
    return lista
```

Ordenamiento por Inserción (Insertion Sort)

```
def insertion_sort(lista):  
    for i in range(1, len(lista)):  
        key = lista[i]  
        j = i - 1  
        while j >= 0 and key < lista[j]:  
            lista[j+1] = lista[j]  
            j -= 1  
        lista[j+1] = key  
    return lista
```

Ordenamiento por Selección (Selection Sort)

```
def selection_sort(lista):  
    n = len(lista)  
    for i in range(n):  
        min_index = i  
        for j in range(i+1, n):  
            if lista[j] < lista[min_index]:  
                min_index = j  
        lista[i], lista[min_index] = lista[min_index], lista[i]  
    return lista
```

Quicksort

```
def quick_sort(lista):  
    if len(lista) <= 1:  
        return lista  
    else:  
        pivot = lista[0]  
        less = [x for x in lista[1:] if x <= pivot]  
        greater = [x for x in lista[1:] if x > pivot]  
        return quick_sort(less) + [pivot] + quick_sort(greater)
```

Merge Sort

```
def merge_sort(lista):  
    if len(lista) <= 1:  
        return lista  
  
    mid = len(lista) // 2  
  
    left_half = lista[:mid]  
    right_half = lista[mid:]  
  
    left_sorted = merge_sort(left_half)  
    right_sorted = merge_sort(right_half)  
  
    return _merge(left_sorted, right_sorted)  
  
def _merge(left, right):  
    merged = []  
  
    i, j = 0, 0  
  
    while i < len(left) and j < len(right):  
        if left[i] < right[j]:  
            merged.append(left[i])  
            i += 1  
        else:  
            merged.append(right[j])  
            j += 1  
  
    merged.extend(left[i:])  
    merged.extend(right[j:])  
  
    return merged
```

4. Metodología Utilizada

El desarrollo de este trabajo integrador se basó en una metodología que cubrió investigación, diseño, implementación y análisis.

4.1. Investigación Previa

La fase inicial consistió en la investigación del tema, utilizando principalmente el material didáctico del aula virtual (PDFs y material audiovisual) y el archivo .ipynb de sintaxis proporcionado por la cátedra. Estas fuentes nos sirvieron para comprender los conceptos teóricos de los algoritmos de búsqueda y ordenamiento y para establecer la base del Marco Teórico.

4.2. Diseño y Desarrollo del Código

Se planteó un caso práctico para simular la gestión de datos. El diseño del código en Python se centró en la modularidad, separando las implementaciones de los algoritmos (algoritmos.py) de la lógica de pruebas y análisis ([analisis.py](#)). Se implementaron funciones auxiliares para generar datos aleatorios y medir el tiempo de ejecución con `time.perf_counter()`.

4.3. Pruebas y Análisis de Resultados

Se ejecutaron pruebas de rendimiento con listas de diversos tamaños, desde pequeñas hasta muy grandes, para cada algoritmo de búsqueda y ordenamiento. Los tiempos obtenidos se recolectaron y analizaron para comparar la eficiencia práctica.

4.4. Herramientas y Recursos

Para la implementación se utilizó Python, con las librerías `time` y `random`. El control de versiones y la colaboración se gestionaron mediante Git.

4.5. Trabajo Colaborativo

El proyecto fue realizado en conjunto entre nosotros. Las tareas se distribuyeron equitativamente, abarcando desde la investigación y redacción del marco teórico hasta la implementación del código, la ejecución de pruebas y el análisis de resultados, utilizando Git para una colaboración fluida.

5. Resultados Obtenidos

El caso práctico permitió validar la eficiencia de los algoritmos de búsqueda y ordenamiento. Implementamos y probamos exitosamente:

Búsqueda: Lineal, Binaria e Interpolación.

Ordenamiento: Burbuja, Selección, Inserción, Quick Sort y Merge Sort.

5.1. Evaluación de Rendimiento

Las pruebas con diferentes tamaños de listas (hasta 10.000 para ordenamiento y 10.000.000 para búsqueda) confirmaron las teorías:

Ordenamiento: Los algoritmos $O(n^2)$ (Burbuja, Selección, Inserción) mostraron un rendimiento muy lento con el aumento de datos. Los $O(n \log n)$ (Quick Sort, Merge Sort) fueron significativamente más rápidos y escalables, demostrando su superioridad para grandes volúmenes.

Búsqueda: La Lineal ($O(n)$) fue la menos eficiente. La Binaria ($O(\log n)$) y la de Interpolación ($O(\log(\log n))$ promedio) fueron muy eficientes, con un crecimiento mínimo del tiempo. La Búsqueda por Interpolación, en particular, exhibió un rendimiento marginalmente superior en las listas probadas, debido a la distribución uniforme de los datos generados.

Estos resultados, visibles en los gráficos del Caso Práctico, validan directamente la Notación Big O y la importancia de la elección algorítmica.

5.2. Dificultades y Correcciones

Se enfrentaron y corrigieron errores en la lógica de `busqueda_binaria` (precedencia de operadores) y `busqueda_interpolacion` (sintaxis y lógica). Ajustar los tamaños de las listas de prueba fue clave para visualizar correctamente las diferencias de rendimiento entre los algoritmos de diversas complejidades.

5.3. Capturas de los resultados

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

```
PS C:\Users\Daniela Bonetti\OneDrive\Desktop\INTEGRADOR-PROGRAMACION> & "rs/Daniela Bonetti/OneDrive/Desktop/INTEGRADOR-PROGRAMACION/trabajo-integrador-programacion/analisis.py"
Comparación de Algoritmos de Búsqueda (Elementos Existentes)
Nota: Para búsqueda binaria e interpolación, las listas se ordenan previamente.
Probando con tamaño de lista: 1000
  Búsqueda Lineal: 0.000007 segundos
  Búsqueda Binaria: 0.000004 segundos
  Búsqueda por Interpolación: 0.000003 segundos
Probando con tamaño de lista: 5000
  Búsqueda Lineal: 0.000043 segundos
  Búsqueda Binaria: 0.000004 segundos
  Búsqueda por Interpolación: 0.000003 segundos
Probando con tamaño de lista: 10000
  Búsqueda Lineal: 0.000236 segundos
  Búsqueda Binaria: 0.000005 segundos
  Búsqueda por Interpolación: 0.000005 segundos
Probando con tamaño de lista: 50000
  Búsqueda Lineal: 0.000050 segundos
  Búsqueda Binaria: 0.000004 segundos
  Búsqueda por Interpolación: 0.000004 segundos
```

Comparación de Algoritmos de Ordenamiento

```
tamaño de la lista: 1000
  Bubble Sort: 0.058339 segundos
  Insertion Sort: 0.024785 segundos
  Selection Sort: 0.051637 segundos
  Quick Sort: 0.001882 segundos
  Merge Sort: 0.002494 segundos
tamaño de la lista: 5000
  Bubble Sort: 1.927894 segundos
  Insertion Sort: 0.661847 segundos
  Selection Sort: 1.233232 segundos
  Quick Sort: 0.014206 segundos
  Merge Sort: 0.016990 segundos
tamaño de la lista: 10000
  Bubble Sort: 6.490959 segundos
  Insertion Sort: 2.578322 segundos
  Selection Sort: 5.506946 segundos
  Quick Sort: 0.046358 segundos
  Merge Sort: 0.041048 segundos
tamaño de la lista: 50000
```

6. Conclusiones

Este trabajo nos permitió profundizar en los algoritmos de búsqueda y ordenamiento en Python, validando su comportamiento y rendimiento en la práctica.

6.1. Aprendizajes Clave

El mayor aprendizaje fue la validación real de la Notación Big O. Comprobamos cómo la eficiencia algorítmica impacta drásticamente en el tiempo de ejecución con grandes volúmenes de datos. Entendimos por qué algoritmos como Quick Sort, Merge Sort y Búsqueda Binaria/Interpolación son esenciales para la escalabilidad, a diferencia de los métodos $O(n^2)$ que se vuelven lentos rápidamente.

6.2. Utilidad para la Programación

Dominar estos algoritmos es importante para optimizar el código y tomar decisiones en el diseño de software. Este conocimiento nos ayudará en futuros proyectos que manejen datos, mejorando la eficiencia y el rendimiento de las aplicaciones.

6.3. Mejoras y Extensiones Futuras

Podríamos ampliar el trabajo implementando más algoritmos (ej. Heap Sort, Radix Sort, Búsqueda Hash) y podríamos agregar la confección de gráficos de los resultados.

6.4. Dificultades y Resoluciones

Las principales dificultades fueron corregir errores de lógica en la implementación de `busqueda_binaria` y `busqueda_interpolacion`, y ajustar los tamaños de las listas de prueba para visualizar correctamente las diferencias de rendimiento. Estas se resolvieron con depuración.

7. Bibliografía

Para la elaboración de este trabajo, consultamos la información en el aula virtual, con los PDF y los archivos collab para confeccionar la base teórica y práctica necesaria para el análisis e implementación de los algoritmos de búsqueda y ordenamiento.

- [Búsqueda y Ordenamiento en Programación], "Búsqueda y Ordenamiento en Programación".
- [BusquedaOrdenamiento.ipynb].

8. Anexos

8.1 Enlace al Video Explicativo del Proyecto

- **Link de video en Youtube:** <https://youtu.be/LmWDORWYyJE>

8.2 Código Fuente Completo del Proyecto

- **Repositorio:** <https://github.com/Danielabonetti/trabajo-integrador-programacion.git>